UNIVERSIDAD CHAMPAGNAT

Licenciatura en Sistemas de Información
Programación Distribuida
Trabajo Práctico N° 5
De API REST a Microservicios

Modalidad: Grupal - 3 a 5 integrantes

FECHA:
GRUPO:
INTEGRANTES:

UNIVERSIDAD CHAMPAGNAT

CONSIGNAS

1. Objetivo

El objetivo del presente Trabajo Práctico es evolucionar la aplicación desarrollada en los trabajos anteriores hacia una arquitectura basada en microservicios, incorporando además un flujo de trabajo colaborativo con Git.

La primera decisión no será tecnológica sino arquitectónica: antes de separar la aplicación, cada grupo deberá analizar el sistema existente, identificar responsabilidades, detectar acoplamientos, definir límites funcionales y justificar qué funcionalidades deberían pertenecer a cada servicio.

> **OBJETIVO CENTRAL:** No se busca dividir el proyecto en varias carpetas ni crear un servicio por cada entidad. La separación deberá responder a responsabilidades de negocio claras y producir componentes con menor acoplamiento y límites explícitos.

2. Modalidad de trabajo y continuidad del proyecto

El trabajo será grupal. Cada grupo estará conformado por un mínimo de 3 y un máximo de 5 integrantes. La organización del equipo, la distribución de tareas y la integración de cambios forman parte de los objetivos del TP.

La participación individual deberá poder verificarse a partir del historial del repositorio: ramas, commits, Pull Requests, revisiones y resolución de conflictos.

> **REPOSITORIO ÚNICO:** Cada grupo creará un único repositorio compartido que funcionará como repositorio base y evolutivo para este TP y para los trabajos prácticos posteriores. No se deberá crear un repositorio nuevo por cada TP. Las distintas entregas deberán quedar identificadas dentro del mismo historial.

3. Alcance del TP5

En esta etapa se trabajará principalmente sobre los siguientes conceptos:

- Git como herramienta de control de versiones y colaboración.
- Repositorio compartido y trazabilidad del trabajo del equipo.
- Análisis de la API REST existente.
- Identificación de responsabilidades y límites funcionales.
- Detección de acoplamientos y propuesta de desacople.
- Definición de microservicios y propiedad de los datos.
- Diseño de las comunicaciones entre servicios.
- Representación de la arquitectura propuesta y justificación de decisiones.

4. Preparación del entorno colaborativo con Git

4.1 Instalación y verificación

Todos los integrantes deberán tener Git instalado y disponible desde una terminal. Verificar la instalación mediante:

```
git --version
```

4.2 Configuración de identidad

Cada integrante deberá configurar Git utilizando su nombre real y una dirección de correo que permita identificar al autor de los commits.

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"

git config --list
```

4.3 Creación del repositorio base del grupo

El grupo deberá crear el repositorio remoto que utilizará como base para toda la evolución del proyecto. Este repositorio será compartido por todos los integrantes y continuará utilizándose en los próximos trabajos prácticos.

- La rama principal deberá denominarse main.
- Todos los integrantes del grupo deberán contar con acceso al repositorio.
- El grupo deberá acordar una versión funcional de la aplicación previa como punto de partida.
- El historial del repositorio deberá conservarse durante los próximos TPs; no se reiniciará el proyecto creando un repositorio nuevo.
- La versión estable de cada entrega deberá poder identificarse sin perder la continuidad del desarrollo.

4.4 Incorporación de los docentes

Agregar como colaboradores del repositorio a los dos docentes de la cátedra utilizando las siguientes direcciones:

```
alvarezadriel@uch.edu.ar
cortezedgardo@uch.edu.ar
```

El acceso deberá mantenerse durante toda la secuencia de trabajos prácticos que utilicen este repositorio.

4.5 Punto de partida del proyecto

El repositorio deberá comenzar con una versión funcional de la aplicación desarrollada previamente. Si los integrantes parten de implementaciones diferentes, el grupo deberá acordar cuál utilizará como base o integrar las partes necesarias antes de comenzar la separación en microservicios.

Antes de continuar, la versión base deberá poder instalarse, ejecutarse y probarse. El primer objetivo es disponer de un estado conocido y reproducible sobre el cual realizar la migración.

4.6 Archivo .gitignore y datos sensibles

Crear o revisar el archivo .gitignore para evitar versionar dependencias, archivos temporales, configuración local y datos sensibles. Como mínimo deberá contemplar:

```
node_modules/
.env
*.log
```

- No versionar contraseñas, tokens, claves privadas ni credenciales.
- No versionar dependencias que puedan reconstruirse mediante el gestor de paquetes.
- Documentar las variables de entorno requeridas mediante un archivo de ejemplo cuando corresponda, sin incluir valores sensibles.

4.7 Flujo de trabajo con ramas

No se realizará todo el desarrollo directamente sobre main. Cada funcionalidad, corrección o tarea relevante deberá desarrollarse en una rama específica y luego integrarse al proyecto mediante un Pull Request.

```
main
  ├── feature/identificar-servicios
  ├── feature/catalog-service
  ├── feature/inventory-service
  ├── fix/validacion-stock
  └── docs/arquitectura
```

Los nombres anteriores son ilustrativos. Los nombres reales deberán reflejar el trabajo realizado por el grupo.

4.8 Operaciones mínimas de Git

Durante el desarrollo todos los integrantes deberán utilizar y poder explicar las operaciones básicas de Git:

```
git clone
git status
git branch
git switch
git add
git commit
git pull
git push
git merge
```

4.9 Commits y Pull Requests

Los commits deberán representar cambios concretos y utilizar mensajes que permitan entender qué se modificó. Se evitarán mensajes genéricos como "cambios", "final", "prueba" o "ahora sí".

```
Add initial shared repository
Document service boundaries
Extract inventory responsibility
Add service communication
Fix stock validation
```

Cada Pull Request deberá indicar brevemente qué problema resuelve, qué partes del proyecto modifica y cómo fue validado antes de integrarse.

4.10 Identificación de entregas dentro del repositorio único

Como el mismo repositorio continuará utilizándose en los TPs siguientes, cada entrega deberá quedar marcada de manera inequívoca. Se recomienda utilizar tags sobre el commit aprobado para la entrega.

```
git tag tp5-entrega
git push origin tp5-entrega
```

En los trabajos siguientes podrán utilizarse etiquetas equivalentes, por ejemplo tp6-entrega, tp7-entrega, etc. La rama main continuará evolucionando después de cada entrega.

5. Análisis de la aplicación existente

Antes de crear microservicios, el grupo deberá construir una visión común de la aplicación actual. El objetivo es comprender qué hace el sistema antes de decidir cómo dividirlo.

Realizar un relevamiento de las funcionalidades existentes e identificar, como mínimo:

- recursos y entidades principales;
- endpoints disponibles y operaciones que realizan;
- reglas de negocio y validaciones;
- módulos, controladores o funciones involucradas;
- acceso a persistencia;
- dependencias entre funcionalidades;
- información compartida por diferentes partes de la aplicación.

Presentar un inventario inicial utilizando una tabla equivalente a la siguiente:

Funcionalidad / endpoint	Responsabilidad actual	Datos utilizados	Dependencias
...	...	...	...
...	...	...	...
...	...	...	...

6. Identificación de responsabilidades

A partir del relevamiento anterior, agrupar las funcionalidades según la responsabilidad que cumplen dentro del sistema. Para cada grupo funcional responder las siguientes preguntas:

1. ¿Qué problema o necesidad del sistema resuelve?
2. ¿Qué operaciones realiza?
3. ¿Qué reglas de negocio contiene?
4. ¿Qué información necesita para trabajar?
5. ¿Qué información crea o modifica?
6. ¿Qué otras funcionalidades necesita conocer?
7. ¿Qué cambios funcionales suelen ocurrir juntos?
8. ¿Podría evolucionar de manera independiente del resto de la aplicación?

CRITERIO: Las responsabilidades deberán describirse en términos funcionales. Evitar definiciones basadas exclusivamente en archivos, controladores, tablas o detalles de implementación.

7. Identificación de acoplamientos

Seleccionar al menos tres situaciones concretas de la aplicación actual donde exista un acoplamiento relevante. Puede tratarse de módulos que comparten lógica, acceso directo a datos ajenos, funciones con múltiples responsabilidades o cambios que obligan a modificar varias partes del sistema.

Para cada caso indicar:

- qué componentes o funcionalidades se encuentran acoplados;
- por qué existe ese acoplamiento;
- qué problema puede generar al crecer o modificar el sistema;
- cómo podría reducirse mediante una separación de responsabilidades.

Caso	Acoplamiento detectado	Problema generado	Propuesta de desacople
1	...	...	...
2	...	...	...
3	...	...	...

8. Propuesta de microservicios

A partir de las responsabilidades y acoplamientos identificados, el grupo deberá proponer una primera división de la aplicación en microservicios. La propuesta deberá contener como mínimo tres servicios claramente diferenciados.

> **IMPORTANTE:** Una entidad, una tabla o un conjunto de endpoints no constituyen por sí solos un límite de microservicio. Tampoco se evaluará positivamente una división excesiva cuyo único resultado sea aumentar las comunicaciones entre componentes.

8.1 Ficha de definición de cada microservicio

Por cada microservicio propuesto completar una ficha con los siguientes elementos:

Elemento	Definición requerida
Nombre	Nombre claro que refleje la responsabilidad del servicio.
Responsabilidad principal	Problema funcional del que será responsable.
Operaciones	Acciones o casos de uso que resolverá.
Reglas de negocio	Validaciones y decisiones que pertenecen al servicio.
Datos propios	Información cuya creación y modificación serán responsabilidad del servicio.
Endpoints	Endpoints existentes que migran al servicio y endpoints nuevos que resulten necesarios.
Dependencias	Otros servicios que necesita consultar o invocar.
Fuera de alcance	Responsabilidades que explícitamente no pertenecen al servicio.
Justificación	Motivo por el cual el límite propuesto reduce acoplamiento o mejora la cohesión.

8.2 Tabla general de responsabilidades

Consolidar la propuesta en una tabla que permita observar la distribución completa del sistema:

Microservicio	Responsabilidad	Datos administrados	Funciones principales	Dependencias
Servicio A	...	...	...	...
Servicio B	...	...	...	...
Servicio C	...	...	...	...

9. Propiedad y acceso a los datos

Para cada conjunto de datos relevante deberá definirse un servicio responsable. El objetivo es evitar que diferentes microservicios modifiquen libremente la misma información o dependan de conocer detalles internos de persistencia de otros servicios.

Completar una matriz equivalente a la siguiente:

Dato / recurso	Servicio propietario	Quién puede consultarlo	Quién puede modificarlo	Forma de acceso
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

> **REGLA DE DISEÑO:** Un servicio no debería modificar directamente los datos cuya responsabilidad pertenece a otro servicio. Si necesita producir un cambio, deberá solicitarlo a través de la interfaz pública definida por el servicio propietario.

10. Comunicación entre microservicios

Una vez definidos los límites, identificar las comunicaciones realmente necesarias entre servicios. En esta etapa se utilizará comunicación HTTP/REST para representar estas interacciones.

Origen	Destino	Necesidad	Método / endpoint	Datos intercambiados	Respuesta esperada
...	...	...	...	...	...
...	...	...	...	...	...

Para cada comunicación propuesta analizar además:

- ¿Es realmente necesario que un servicio conozca al otro?
- ¿Se está duplicando una regla de negocio que debería tener un único propietario?
- ¿La información solicitada podría pertenecer al servicio que la necesita?
- ¿El contrato entre servicios puede expresarse sin conocer detalles internos de implementación?
- ¿Qué ocurre con el caso de uso si el servicio destino no se encuentra disponible?

10.1 Trazado de casos de uso

Seleccionar al menos dos operaciones importantes del sistema y describir su recorrido completo en la arquitectura propuesta.

```
Cliente
  │
  ▼
Servicio A
  │  HTTP
  ▼
Servicio B
  │
  ▼
Respuesta
```

El recorrido deberá indicar quién inicia la operación, qué servicio toma cada decisión, qué datos se consultan y qué servicio es responsable de modificar el estado.

11. Diagrama de arquitectura

Realizar un diagrama de la arquitectura propuesta. El diagrama deberá mostrar como mínimo:

- los microservicios definidos;
- la responsabilidad general de cada servicio;
- las comunicaciones entre servicios y su sentido;
- los principales puntos de entrada desde el cliente;
- la persistencia o datos bajo responsabilidad de cada servicio.

No se evaluará la herramienta gráfica utilizada. Se evaluará que el diagrama represente de manera coherente las decisiones descriptas en el resto del documento.

12. Validación de los límites propuestos

Antes de comenzar la implementación, revisar cada servicio propuesto utilizando el siguiente checklist:

☐ Puede explicarse su responsabilidad principal en una frase clara.

☐ Agrupa reglas de negocio relacionadas entre sí.

☐ Posee un conjunto de datos o decisiones sobre los cuales es responsable.

☐ No fue creado únicamente porque existe una tabla, entidad o controlador con ese nombre.

☐ Su interfaz pública puede definirse sin exponer detalles internos.

☐ Puede modificarse internamente sin exigir cambios simultáneos en todos los demás servicios.

☐ No duplica reglas de negocio cuya autoridad pertenece a otro servicio.

☐ Las dependencias con otros servicios son necesarias y están justificadas.

☐ El grupo puede explicar qué responsabilidades quedan explícitamente fuera del servicio.

Si dos servicios necesitan compartir continuamente sus datos internos, modificar las mismas entidades o coordinar casi todas sus operaciones, el grupo deberá revisar si la separación propuesta es adecuada.

13. Estructura inicial del repositorio

La estructura definitiva dependerá de la arquitectura elegida. Sin embargo, al finalizar esta etapa el repositorio deberá permitir identificar claramente la documentación de arquitectura, la aplicación base y el trabajo futuro de cada servicio.

```
proyecto-distribuido/
├── README.md
├── docs/
│   ├── arquitectura.md
│   └── diagrama-arquitectura.*
├── <aplicacion-base-o-servicios>/
├── .gitignore
└── ...
```

No se exige utilizar exactamente esta estructura. El grupo deberá mantener una organización coherente y documentarla en el README del repositorio.

14. Entregables de esta etapa

La revisión de esta etapa deberá permitir comprobar tanto la preparación del trabajo colaborativo como la calidad de las decisiones arquitectónicas. Se deberá presentar:

1. URL del repositorio único del grupo.
2. Listado de integrantes.
3. Docentes agregados como colaboradores: alvarezadriel@uch.edu.ar y cortezedgardo@uch.edu.ar.
4. Versión funcional utilizada como punto de partida.
5. Archivo .gitignore y configuración de variables sensibles.
6. Evidencia de uso de ramas, commits y al menos un Pull Request.
7. Inventario de funcionalidades y endpoints de la aplicación existente.
8. Identificación de responsabilidades.
9. Al menos tres ejemplos concretos de acoplamiento y su propuesta de desacople.
10. Propuesta de al menos tres microservicios.
11. Ficha completa y justificación de cada microservicio.
12. Matriz de propiedad de datos.
13. Matriz de comunicaciones entre servicios.
14. Trazado de al menos dos casos de uso.
15. Diagrama de arquitectura.

16. Checklist de validación de límites revisado por el grupo.

15. Criterios generales de revisión

En esta etapa no existe necesariamente una única división correcta del sistema. Se evaluará principalmente la capacidad del grupo para justificar las decisiones tomadas y mantener coherencia entre responsabilidades, datos, comunicaciones y arquitectura.

- Claridad en la identificación de responsabilidades.
- Calidad de la justificación de los límites de cada servicio.
- Reducción del acoplamiento y cohesión interna de los servicios.
- Definición clara de la propiedad de los datos.
- Coherencia de las comunicaciones entre servicios.
- Trazabilidad del trabajo colaborativo mediante Git.
- Consistencia entre documentación, diagrama y propuesta técnica.

RESULTADO ESPERADO: Al finalizar esta etapa, el grupo deberá contar con un repositorio colaborativo operativo y una arquitectura de microservicios suficientemente justificada como para comenzar la migración del código sin separar funcionalidades de manera arbitraria.